

Lab: Public Key Infrastructure (PKI) Using OpenSSL

Estimated time: 60 minutes

Overview

This lab introduces you to the practical implementation of a Public Key Infrastructure (PKI) using OpenSSL. You will create a self-signed Certificate Authority (CA), generate and sign certificates, and verify them. These concepts form the backbone of secure communications in systems like HTTPS and VPNs.

Learning Objectives

By the end of this lab, you will be able to:

- Create a self-signed Certificate Authority (CA)
- Generate and sign certificates for servers or clients
- Verify certificates using the CA

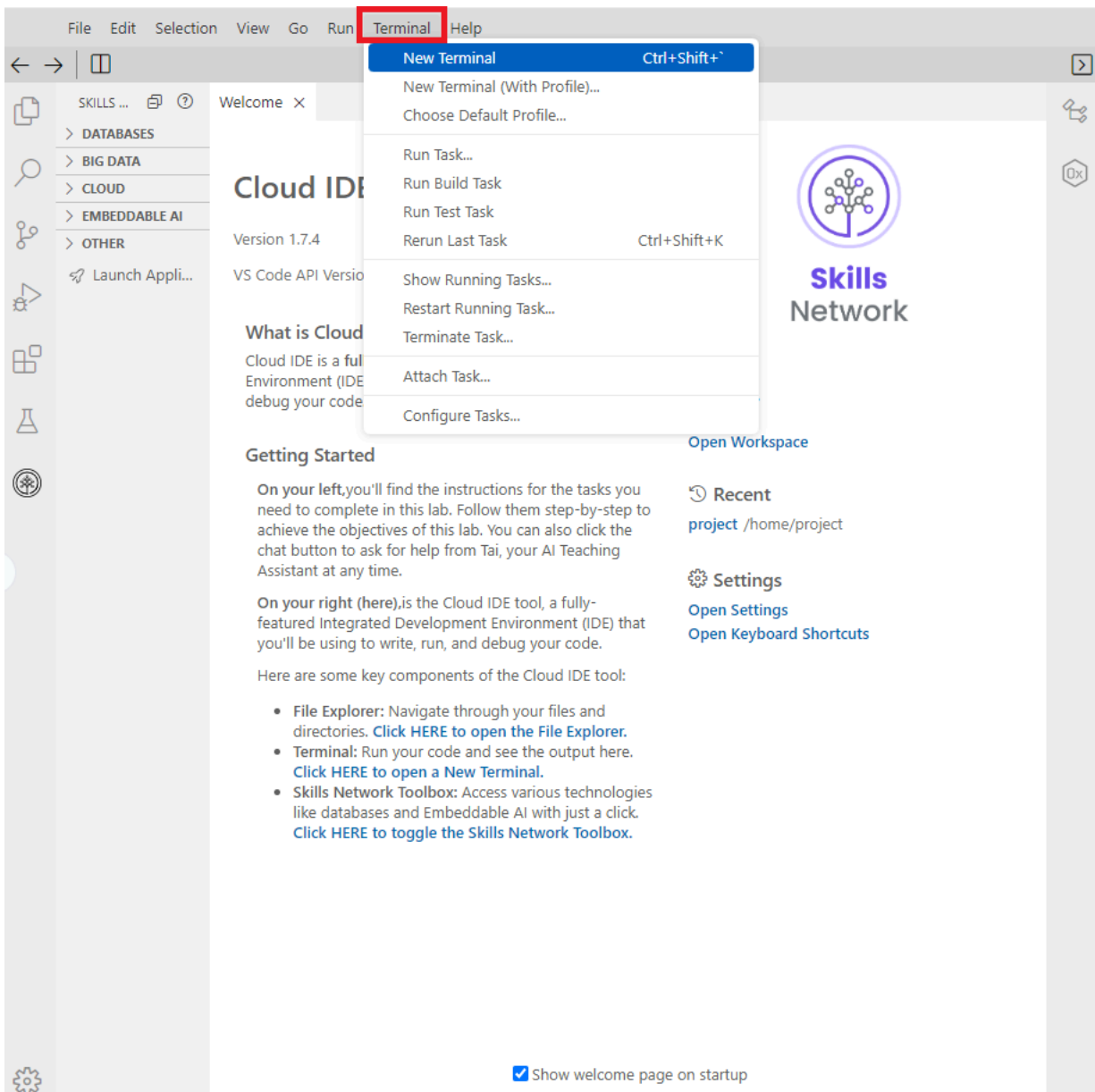
Prerequisites

- Familiarity with the Linux command line
- Basic understanding of cryptographic concepts such as public or private keys and digital certificates

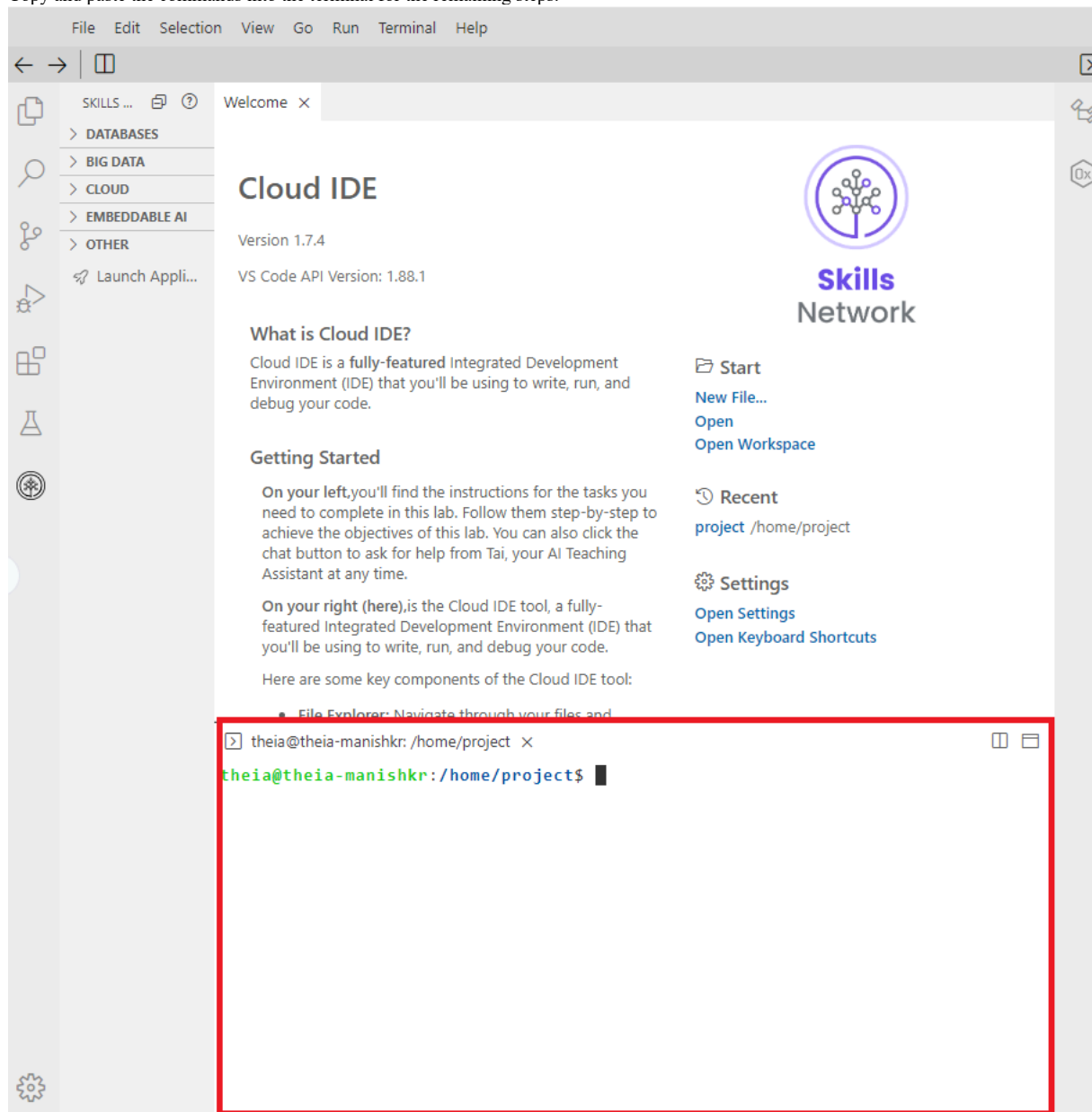
Initializing the Lab Environment

Let's first look at the steps to initialize the lab environment.

- Select **Terminal** and then select **New Terminal** from the drop down.



- Copy and paste the commands into the terminal for the remaining steps.



Step 1: Create a Certificate Authority (CA)

Step A: Generate the private key for the CA

```
openssl genpkey -algorithm RSA -out ca.key -aes256 -pass pass:yourpassword
```

Command Breakdown:

- `openssl genpkey -algorithm RSA`: Generates an RSA private key.
- `-out ca.key`: Saves the private key to `ca.key`.
- `-aes256`: Encrypts the private key with AES-256 encryption.
- `-pass pass:yourpassword`: Specifies the password for encrypting the private key.

Step B: Generate the CA Certificate

```
openssl req -new -x509 -key ca.key -sha256 -days 365 -out ca.crt -passin pass:yourpassword
```

Command Breakdown:

- `openssl req -new -x509`: Creates a new certificate and self-signs it as a CA certificate.
- `-key ca.key`: Specifies the CA private key.
- `-sha256`: Uses the SHA-256 hash algorithm.
- `-days 365`: Sets the certificate's validity period to 365 days.
- `-out ca.crt`: Saves the certificate to `ca.crt`.

Next, fill the certificate details, such as Country Name and Common Name when prompted.

Step 2: Generate a Private Key for the Server

Copy and paste the below command to generate private key for the server.

```
openssl genpkey -algorithm RSA -out server.key
```

This command generates the private key for the server and saves it as `server.key`.

Step 3: Create a Certificate Signing Request (CSR) for the Server

Enter the below command to create a CSR for the server.

```
openssl req -new -key server.key -out server.csr
```

Command Breakdown:

- `-key server.key`: Specifies the server's private key.
- `-out server.csr`: Saves the CSR to `server.csr`.

Now, provide the necessary details for the CSR, such as Common Name (usually the server's hostname).

Step 4: Sign the Server Certificate with the CA

Enter the below command for signing the server certificate with the CA.

```
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -days 365 -sha256 -passin pass:yourpassword
```

Command Breakdown:

- `openssl x509 -req`: Processes the CSR for signing.
 - `-in server.csr`: Specifies the server CSR file.
 - `-CA ca.crt`: Specifies the CA certificate.
 - `-CAkey ca.key`: Specifies the CA private key.
 - `-CAcreateserial`: Creates a serial number file for the certificate.
 - `-out server.crt`: Saves the signed certificate to `server.crt`.
-

Step 5: Verify the Server Certificate

Enter the below command to verify the server certificate.

```
openssl verify -CAfile ca.crt server.crt
```

Command Breakdown:

- `openssl verify`: Verifies a certificate.
- `-CAfile ca.crt`: Specifies the CA certificate to verify against.
- `server.crt`: Specifies the server's certificate.

If the verification is successful, OpenSSL will provide an output `server.crt: OK`.

Step 6: Simulate a Client-Server Authentication

Follow the below steps to simulate a client-server authentication.

Step A: Use the server certificate

Start a simple server using the server key and certificate entering the below command.

```
openssl s_server -cert server.crt -key server.key -accept 8443
```

Step B: Connect as a client

From another terminal, connect to the server as a client using the below command.

```
openssl s_client -connect localhost:8443 -CAfile ca.crt
```

Further, inspect the certificate details displayed in the client terminal.

Exercises

Exercise 1: Set Up a Basic Certificate Authority (CA)

Objective: Learn to set up a simple CA, which is the foundation of any PKI infrastructure. This exercise helps you understand how certificates are issued and managed by a CA.

Task:

1. Create a directory structure for your CA, which includes directories for the CA's certificates, private keys, and database.
2. Generate a root private key for the CA.
3. Create the root certificate (self-signed) for the CA.

Steps:

1. Create directories for the CA:

```
mkdir -p ~/myCA/newcerts ~/myCA/private  
touch ~/myCA/myCAindex  
chmod 700 ~/myCA/private  
chmod 755 ~/myCA/newcerts
```

2. Generate the root private key for your CA:

```
openssl genpkey -algorithm RSA -out ~/myCA/private/myCAkey.pem -pkeyopt rsa_keygen_bits:2048
```

3. Generate the root certificate for your CA:

```
openssl req -new -x509 -key ~/myCA/private/myCAkey.pem -out ~/myCA/myCAcert.pem -days 3650
```

Hint:

- Ensure that your CA root certificate (myCAcert.pem) is kept secure because it is the foundation of your trust chain. The root certificate is used to sign other certificates in the PKI.

Exercise 2: Issue Certificate Using Your Own CA

Objective: Simulate the process of issuing a certificate using a Certificate Authority (CA) that you've set up in Exercise 1. Understand how an intermediate certificate is created and signed by the root CA.

Task:

1. Create a private key for user1.
2. Generate a CSR for user1 using the private key.
3. Use the root CA that you've created to issue a certificate for user1.

Steps:

1. Create a private key for user1:

```
openssl genpkey -algorithm RSA -out user1_private_key.pem -pkeyopt rsa_keygen_bits:2048
```

2. Create a CSR for user1:

```
openssl req -new -key user1_private_key.pem -out user1_csr.pem
```

3. Use the root CA to sign user1's CSR and issue a certificate:

```
openssl ca -in user1_csr.pem -out user1_cert.pem -cert ~/myCA/myCAcert.pem -keyfile ~/myCA/private/myCAkey.pem -days 365
```

Hint:

- The root CA's private key signs user1's CSR, issuing a certificate that can be trusted because it is signed by the root certificate.
- Remember to review the contents of the CSR and the certificate using the `openssl req -text -noout -verify -in` and `openssl x509 -text -noout` commands.

Steps to Fix the Error

If you encounter the './demoCA/newcerts is not a directory' error, follow the steps given below:

1. Create the necessary directory structure:

```
mkdir -p demoCA/newcerts
mkdir -p demoCA/private
touch demoCA/index.txt
echo 1000 > demoCA/serial
```

- **demoCA/newcerts**: Directory to store issued certificates.
- **demoCA/private**: Directory to store private keys.
- **demoCA/index.txt**: Database file for issued certificates.
- **demoCA/serial**: File containing the starting serial number for certificates (e.g., 1000).

2. Modify the OpenSSL configuration file:

If you prefer to use your custom directory (e.g., ~/myCA), update the `openssl.cnf` file. Locate your OpenSSL configuration file (`/usr/lib/ssl/openssl.cnf` or `/etc/ssl/openssl.cnf`) and modify the `dir` directive.

Example for modification:

```
[ CA_default ]
dir = /path/to/your/CA
```

Or specify the configuration explicitly in your command:

```
openssl ca -in user1_csr.pem -out user1_cert.pem -cert ~/myCA/myCAcert.pem -keyfile ~/myCA/private/myCAkey.pem -days 365 -config /usr/lib/ssl/o
```

3. Re-run the command:

```
openssl ca -in user1_csr.pem -out user1_cert.pem -cert ~/myCA/myCAcert.pem -keyfile ~/myCA/private/myCAkey.pem -days 365
```

These steps should resolve the demoCA/newcerts error and allow OpenSSL to issue the certificate successfully.

Conclusion

In this lab, you learned how to create and manage a simple PKI using OpenSSL. By establishing a CA, signing server certificates, and verifying them, you've explored the foundations of secure communication in PKI. This knowledge is critical for understanding how real-world systems, such as HTTPS, authenticate and encrypt communications.

Author(s)

Dr. Manish Kumar

© IBM Corporation. All rights reserved.